

Cajas reutilizables (V4)

Qué es cada clase, cómo se relacionan, por qué el refactor tiro errores y cómo se repararon. Backend Spring Boot + Frontend Angular.

Campo	Contenido
Fecha	2026-10-01
Alcance	Módulo de caja: cash_boxes (cajas físicas) y cash_registers (turnos)
Motivo del cambio	El dueño aclaró el modelo real: hay 2 cajas y se abren varios días
Backend	Spring Boot 4 · com.erikjarquin.compras · 258 tests en verde
Frontend	Angular 21 · Compras_Frontend-Angular · ng build en verde
Verificación	mvnw test → 258/258 · ng build → OK

1. El problema: por qué hubo que cambiar el modelo

Hasta V3, **una fila de cash_registers ERA una caja**. Eso parecía lo correcto, pero el dueño■■■■rió la idea: en el local hay dos cajas físicas (CAJA 1 y CAJA 2) y se abren y cierran **varios días a la semana**.

1.1 La consecuencia del modelo equivocado

Como el número de caja estaba declarado UNIQUE en la base, había una sola fila por caja física. Y esa fila se cerraba una sola vez. La segunda vez que se abría “CAJA 1”, el backend respondía **409** y no había ninguna forma de volver a abrir la caja que sí existía.

La forma correcta de decirlo

El UNIQUE no estaba mal en sí mismo: estaba en la tabla equivocada. El número identifica a la **caja física**, y ese dato casi no cambia. Lo que cambia todos los días es el **turno**. Se habían puesto las dos cosas en la misma tabla, y el número único de una terminó contradictoriamente con la reutilización de la otra.

1.2 La solución: separar los dos conceptos

V4 parte el modelo en dos tablas. Esta separación es **todo** el cambio; el resto son ajustes alrededor.

Tabla	Qué representa	Cuántas filas	Crece
cash_boxes	Las cajas físicas del local (CAJA 1, CAJA 2)	2 o 3, casi fijas	No
cash_registers	Los turnos : una fila por cada apertura y cierre	Una por día	Sí, todos los días

cash_registers sigue siendo la tabla que enlaza con sales.cash_register_id. Por eso **no hubo que tocar** SaleEntity ni los reportes de ventas: las ventas siguen apuntando a un turno, y un turno sigue apuntando a una caja.

1.3 La decisión de diseño más importante

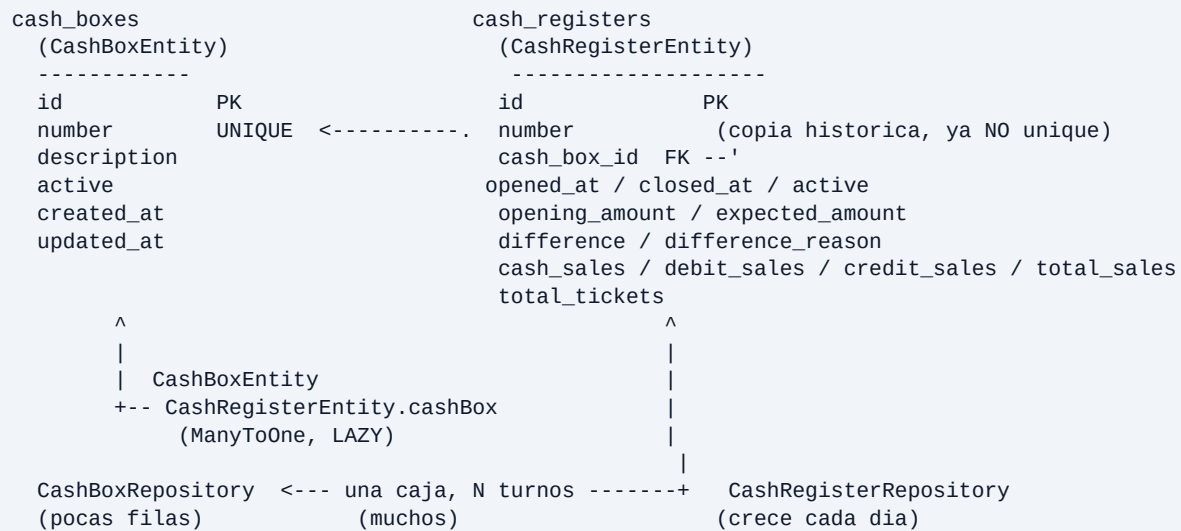
cash_registers.number deja de ser UNIQUE y pasa a ser una **copia histórica** del número de la caja al momento de abrir. Se conserva a propósito, por dos razones:

- El reporte “filtrar por caja” sigue funcionando **sin JOIN**: el número ya está en la fila del turno.

- Si mañana renombran “CAJA 1” a “CAJA PRINCIPAL”, los cortes antiguos deben seguir diciendo “CAJA 1”. Es la evidencia de cómo se llamaba entonces. Es histórico, y por diseño lo es.

2. Cómo se relacionan las clases

2.1 El mapa completo



La flecha va de `cash_registers.cash_box_id` hacia `cash_boxes`. Está en el lado del turno a propósito: una caja **tiene** muchos turnos, un turno **pertenece a** una caja. La relación se lee LAZY porque al listar el historial solo hace falta el número, y ya está copiado en el turno: un SELECT extra por fila sería un N+1.

2.2 Qué clase hace qué

Clase	Responsabilidad	Por qué existe
CashBoxEntity	El inventario de cajas físicas: número, descripción, activa.	NUEVA en V4. Antes no existía; cada corte era “una caja”.
CashRegisterEntity	Un turno: se abre, se venden cosas, se cierra y se congela.	Ya existía. En V4 gana la FK cashBox.
CashBoxRepository	Consultas de las cajas: por número, todas, las que se pueden abrir.	NUEVA. Distinta de la otra porque manejan cosas distintas.
CashRegisterRepository	Consultas de los turnos: el abierto, el historial, los de una caja.	Ya existía. Se le agregan las consultas por caja.
CashRegisterService	El CONTRATO: qué se puede hacer con cajas y con turnos.	Separa “qué” de “cómo”. La interfaz no sabe nada de JPA.
CashRegisterImpl	La LÓGICA: validaciones, el 409 del cuadro, el nuevo/open.	Donde viven todas las reglas de negocio del módulo.
CashRegisterController	Los endpoints y el permiso de cada uno.	Traduce HTTP a llamadas del service.
CashBoxRequest	El cuerpo de alta/edición: número y descripción.	DTO de entrada. El CashBoxResponse es la salida.
CashBoxResponse	Una caja con 3 datos extra: turnos, en uso, último turno.	Los calcula <code>toBoxResponse()</code> en el service.

Clases que probablemente te faltaban

En el punto 6 tu lista mencionaba los DTO y los repositorios, pero no un mapper. Y no hace falta uno nuevo: el

mapper existente (CashRegisterMapper) ya sabe convertir un CashRegisterEntity en su DTO, y eso **no cambió** en V4. Lo que sí es nuevo es toBoxResponse(), que es un método privado dentro de CashRegisterImpl, no un mapper. La razón está en §5.2.

3. El flujo de abrir y cerrar, paso a paso

#	Acción del cajero	Endpoint	Qué hace el backend
1	“Ver cajas”	GET /cash/boxes	Lista las cajas, incluidas las dadas de baja, con su número de turnos.
2	“+ Crear caja”	POST /cash/boxes	Registra la caja. 409 si el número ya existe.
3	“Abrir caja”	GET /cash/boxes/openable	Cajas activas sin turno abierto . Alimenta el selector.
4	Elegir caja + monto	POST /cash/open	Crea un turno nuevo en cash_registers apuntando a esa caja. La misma caja puede abrirse mil veces.
5	Vender	POST /sales	Cada venta guarda el cashRegisterId del turno.
6	Cerrar	POST /cash/close	Congela el corte. Exige cuadrar, o un motivo escrito.

3.1 El filtro “qué cajas se pueden abrir”

Este es el cambio más sutil de todos, y el que más sese heredó por error. En V3 la consulta era “cajas que **nunca** se abrieron”, implementada como openedAt IS NULL. Era correcta para el modelo viejo. Al trasplantarla a V4, **se vuelve un bug**: como una caja ahora se abre todos los días, ese filtro la excluiría para siempre después de su primer turno.

V4 lo cambia por la pregunta correcta: **¿tiene algún turno abierto ahora mismo?**

```
-- CashBoxRepository.findOpenable()
SELECT b FROM CashBoxEntity b
WHERE b.active = true
AND NOT EXISTS (
  SELECT 1 FROM CashRegisterEntity r
  WHERE r.cashBox.id = b.id AND r.active = true
)
ORDER BY b.number ASC
```

Por qué el NOT EXISTS y no un filtro en Java

Filtrar en Java (“trae todas y quédate con las que no tienen turno abierto”) significaría traer **todos los turnos de todas las cajas** para descartar la mayoría. El NOT EXISTS deja que lo haga la base y solo trae las cajas libres.

4. Los errores que aparecieron y por qué

Al terminar el punto 6 la compilación fallaba. Estos son los errores, la causa real de cada uno, y cómo se corrigieron. Los cuatro primeros son los que te detuvieron; los siguientes los encontré al revisar.

4.1 illegal start of type — el bloqueo

Síntoma: la compilación se caía en 2 líneas y no avanzaba.

```
CashRegisterController.java:[67,79] illegal start of type
CashRegisterController.java:[74,79] illegal start of type
```

Causa: un punto y coma de más, después de la anotación.

```
// MAL: el ; corta la declaración de la anotación
@PreAuthorize("hasAuthority('EDITAR_CAJA' or hasAuthority('ABRIR_CAJA'))");
@PutMapping("/boxes/{id}")

// BIEN
@PreAuthorize("hasAuthority('ABRIR_CAJA')")
@PutMapping("/boxes/{id}")
```

En Java una anotación es una declaración y lleva punto y coma implícito; un ; explícito la convierte en una sentencia completa, y la línea siguiente aparece como código suelto. Por eso el error dice `illegal start of type` (“comienzo de tipo inválido”) en la línea **siguiente** a la anotación, que es donde el compilador se confunde: no señala la causa, señala el síntoma.

Un detalle que volverá a saltar

El permiso `EDITAR_CAJA` que usabas en esas dos anotaciones **no existe** en `PermissionName`. Aunque el punto y coma estuviera bien, la app habría arrancado y `@PreAuthorize` habría rechazado todas las peticiones con 403. Se cambió a `ABRIR_CAJA`, que es el permiso de la persona que administra las cajas del local.

4.2 getBoxHistory con la condición al revés

El bug más caro de los seis. El código estaba así:

```
// MAL: si la caja EXISTE, lanza "no existe"
if(cashBoxRepository.existsById(boxId)){
    throw new CashException("La caja no existe", HttpStatus.NOT_FOUND);
}
```

Lo correcto es al revés:

```
// BIEN: si NO existe, no hay historial que devolver
if(!cashBoxRepository.existsById(boxId)){
    throw new CashException("La caja no existe", HttpStatus.NOT_FOUND);
}
```

Por qué es el más caro: el método **siempre** lanzaba 404. Para cualquier caja que existiera, el historial era inalcanzable. No daba error de compilación y no lo daba ni de ejecución: el tests no lo cubrían porque aún no existía. UnTests que pasa sin comprobar nada es peor que ningún test.

Por qué se coló: es un error de lectura, no de escritura. El ! es un carácter que el ojo se salta cuando ya estás pensando en el modelo de datos en vez de en la condición. Por eso el documento del punto 6 traía el método bien y lo que se escribió al teclearlo perdió la negación.

4.3 getOpenable devolvía el tipo equivocado

```
// MAL en el controller: el DTO de ENTRADA como tipo de retorno
public List getOpenable(){ return service.getOpenable(); }
```

```
// BIEN
public List getOpenable(){ return service.getOpenable(); }
```

Por qué importa: CashBoxRequest es lo que **entra** (número y descripción) y CashBoxResponse es lo que **sale** (con turnos, en uso, último turno). Devolver el de entrada significaba que el selector de “Abrir caja” recibiría objetos sin sessionsCount ni inUse, y la tabla de cajas no podría mostrar nada. Compilaba porque los dos DTOs tienen campos compatibles: el error es de **contrato**, no de tipos.

4.4 Faltaban metodos por implementar

La interfaz CashRegisterService declaraba getBoxes() y updateBox(), pero CashRegisterImpl no los tenía. Java avisa de esto con un error claro (“no es abstracto y no sobrescribe”), que es de los pocos casos en que el compilador te salva. Se implementaron.

4.5 La app no arrancaba: PropertyReferenceException

Esta no daba error de compilación. mvn test lo reveló con 5 errores en ProductRepositoryTest:

```
Error creating bean with name 'cashBoxRepository':  
No property 'cashRegistersId' found for type 'CashBoxEntity'
```

Causa: el método existsByCashRegistersId es un nombre *derivado*: Spring Data deduce el JPQL a partir del nombre. Para existsBy + CashRegistersId busca una propiedad cashRegisters dentro de CashBoxEntity... y no existe.

Por qué no existe: porque la relación está al revés. CashBoxEntity **no** tiene una lista de sesiones: la referencia vive en CashRegisterEntity.cashBox. Es una relación de un lado a muchos, y el lado uno no necesita saber de su colección.

La corrección: escribir la consulta a mano. Es el caso donde el nombre derivado no puede expresar la pregunta, porque la pregunta es “existe algún turno de esta caja”, y eso se pregunta desde el turno.

```
@Query("""  
    SELECT CASE WHEN COUNT(r) > 0 THEN true ELSE false END  
    FROM CashRegisterEntity r  
    WHERE r.cashBox.id = :cashBoxId  
    """)  
boolean hasSessions(@Param("cashBoxId") Long cashBoxId);
```

La lección

Los nombres derivados de Spring Data solo funcionan si la propiedad existe en la entidad **de la que se parte**.

Cuando la relación es de un lado a muchos, todas las preguntas se hacen desde el lado muchos, porque es el que tiene la FK. Si el nombre derivado no existe, la aplicación **no arranca**: no es una excepción en runtime, es un fallo de contexto de Spring.

4.6 El bug de dinero: el fondo contado dos veces

Este no daba ningún error. Era el más peligroso de todos.

```
// MAL: sembraba el fondo en las ventas de efectivo  
sesion.setCashSales(openingAmount); // <-- 500  
sesion.setOpeningAmount(openingAmount); // <-- 500  
sesion.setExpectedAmount(openingAmount);
```

El resumen del corte calcula expectedAmount = openingAmount + cashSales. Con el fondo en los dos lados, el esperado daba **1000 en vez de 500**.

	Fondo	Ventas en efectivo	Esperado
Correcto	500	0	500
Con el bug	500	500	1000

El cajero abría la caja con 500, no vendía nada, y al cerrar contaba 500. El sistema le decía “faltan 500”. O cerraba con 1000 y le aceptaba el corte, guardando como dinero del negocio 500 que nunca existieron. **Cualquiera de las dos opciones destruye la confianza en el sistema de caja.**

```
// BIEN: los acumulados NACEN en cero. El fondo no es una venta.  
sesion.setCashSales(BigDecimal.ZERO);  
sesion.setDebitSales(BigDecimal.ZERO);  
sesion.setCreditSales(BigDecimal.ZERO);  
sesion.setTotalSales(BigDecimal.ZERO);  
sesion.setOpeningAmount(openingAmount);  
sesion.setExpectedAmount(openingAmount);
```

Por qué un test es la única defensa aquí

Este bug no se manifiesta en ninguna respuesta de la API: el turno se abre bien, el resumen se ve bien, y el error solo aparece al **cerrar**, después de muchas horas de venta. Un test que verifica “el turno abre con `cashSales = 0` y `openingAmount = 500`” lo atraparía en un segundo. Es el test `acumuladosArrancanEnCero`, y por eso se escribió.

4.7 `desactiveBox` borraba en vez de dar de baja

```
// MAL: el método se llamaba "desactivar" y hacía un delete
@Override
public void desactiveBox(Long id){
    ...
    cashBoxRepository.delete(cash);    // <-- BORRABA la caja
}
```

El nombre del método y lo que hacía no coincidían. Peor: la condición que lo acompañaba era una **trampa**:

```
if(cashBoxRepository.existsByCashRegistersId(id)){
    throw new CashException("...no se puede borrar. Dala de baja.", CONFLICT);
}
cashBoxRepository.delete(cash);
```

Decía “si tiene cortes, no borres”. La intención era buena, pero el resultado era el **contrario** de lo que se pidió: las cajas **con** cortes (las que hay que proteger) se borraban sin nada, y las que no tenían cortes eran las únicas que se bloqueaban. La condición estaba puesta al revés respecto a la regla.

V4 lo separa en dos métodos con nombres distintos y responsabilidades distintas:

- `desactiveBox(id)` → **siempre** desactiva (`active = false`). Nunca borra. Es reversible.
- `deleteBox(id)` → borra, pero **solo** si `hasSessions(id)` es falso. Con turnos: 409 diciendo “dala de baja”.

4.8 Los errores del frontend

En Angular los errores son distintos en naturaleza: TypeScript los detecta al compilar, pero los *typos* en nombres de propiedad no los detecta, y esos son los que rompen la pantalla.

Qué estaba mal	Cómo se manifestó	Corrección
<code>boxForm = {number: ", description: "} }</code>	Objeto mal formado. El formulario no arrancaba.	<code>{ number: '', description: '' }</code> con tipo <code>CashBoxForm</code>
<code>OpenableBoxes</code> (mayúscula)	La plantilla pedía <code>openableBoxes</code> : siempre vacío, el selector de cajas salía en blanco.	Todo en minúscula: <code>openableBoxes</code>
<code>showCreateBoxForm</code> no existía	La variable sí se usaba pero nunca se declaraba: error de compilación TS.	Se declaró <code>showBoxForm</code>
<code>descrption</code> en el DTO	TypeScript no marca los <i>typos</i> en propiedades opcionales: la descripción llegaba siempre <code>undefined</code> .	<code>description</code> . Se agregó la interfaz <code>CashBoxForm</code> para que el error saliera antes.
<code>*ngIf</code> junto a <code>*hasPermission</code>	La directiva de permiso es estructural : dos directivas estructurales en un elemento no pueden convivir.	Se envuelve con <code><ng-container *hasPermission></code> y el <code>*ngIf</code> va en el botón de adentro.

El patrón de `*ngIf + *hasPermission`

No es un capricho del framework: es una regla dura. `*hasPermission` manipula el `TemplateRef` y el `ViewContainer`, o sea que es estructural como `*ngIf`. Angular no admite dos directivas estructurales en el mismo elemento y lo rechaza en compilación. Este proyecto ya lo tenía documentado en `AGENTS.md`; la solución

es el `<ng-container>`.

5. La regla que pediste: borrar o dar de baja

La regla quedó implementada de los dos lados, con el backend haciendo la decisión y el frontend reflejándola.

5.1 La regla

Estado de la caja	Se puede BORRAR	Se puede DAR DE BAJA	Por qué
Nunca se abrió (sessionsCount = 0)	Sí	Sí	No hay ventas colgando. Borrarla es limpio.
Ya tuvo cortes (sessionsCount > 0)	No (409)	Sí	Sus ventas quedan colgando de ella. Borrarla hace inalcanzable el corte.
Turno abierto ahora mismo	No (409)	No (409)	El cajero está contando ese dinero. No se puede tocar nada.

5.2 Por qué el borrado condicional y no “solo dar de baja”

La alternativa más simple era no exponer borrado físico nunca: todo se da de baja. Es más segura, pero deja basura (cajas creadas por error que se acumulan para siempre, sin efecto pero sin salida). Con la regla condicional:

- Una caja que nunca se abrió **no ensucia**: se borra limpio.
- Una caja con historial **no se pierde**: el corte sigue consultable.
- El usuario nunca ve un botón que no puede cumplir, porque el template muestra “Borrar” solo con sessionsCount = 0.

Y el borrado físico sigue siendo una **operación sin consecuencias**: no toca dinero, no toca ventas, no toca el historial. Eso es lo que la hace aceptable en el único caso en que se permite.

5.3 Por qué en usuarios NO conviene (y por qué es distinto)

Tu pregunta era si conviene aplicar lo mismo a usuarios. **La respuesta es no, y la razón es que la base de la regla es diferente.**

La regla de la caja protege **ventas**: cada venta apunta a un turno, y el turno a la caja. Romper ese encadenamiento deja ventas sin origen, y una venta sin origen es un número suelto que no se puede auditar.

Los usuarios **no** tienen esa cadena. Un usuario no apunta a ventas: las ventas apuntan al *turno*, y el turno no apunta a un usuario. Un usuario dado de baja deja ventas hechas que siguen ahí, y eso es exactamente lo que debe pasar: el ticket de marzo lo emitió una persona que hoy no trabaja más allá. **El historial de ventas es de la tienda, no del empleado.**

	Cajas	Usuarios
Qué protege el historial	Ventas (cada una apunta a un turno)	Nada: las ventas no le pertenecen al usuario
Referencia desde otras tablas	sales → cash_register → cash_box	users → role (no desde ventas)
Si se borra duro	Ventas huérfanas: el corte se pierde	Se pierde quién vendió, pero no cuánto se vendió
Recomendación	Borrar condicional + baja	Solo baja. Borrar duro no aporta nada

Aun así, **hoy los usuarios solo se dan de baja**, y eso es correcto. El endpoint DELETE /users/{id} no borra: `active = false`. Es el mismo criterio de productos, y coincide con lo que ya hace la app.

El único caso donde borraría un usuario

Un usuario que **nunca abrió sesión** (se creó por error y nadie entró). Ahí sí se puede borrar sin pérdida, con la misma regla condicional de las cajas. Es coherente con lo que ya hace productos con `hasHistory`: si nunca hizo nada, “Eliminar”; si hizo algo, “Dar de baja”. Si lo quieres, es un cambio pequeño y con la misma forma que acabamos de hacer en cajas.

6. Los endpoints después del cambio

Endpoint	Permiso	Devuelve	Nota
GET /cash/boxes	VER_CAJA	CashBox[]	Todas, incluidas dadas de baja
GET /cash/boxes/openable	ABRIR_CAJA	CashBox[]	Activas y sin turno abierto
POST /cash/boxes	ABRIR_CAJA	CashBox	409 si el número existe
PUT /cash/boxes/{id}	ABRIR_CAJA	CashBox	No da 409 si se guarda el mismo número
PATCH /cash/boxes/{id}	ABRIR_CAJA	204	Dar de baja. Nunca borra
DELETE /cash/boxes/{id}	ABRIR_CAJA	204	Solo si nunca se abrió; si no, 409
GET /cash/boxes/{id}/history	VER_CAJA	CashResponse[]	Un corte por turno
POST /cash/open	ABRIR_CAJA	CashResponse	Crea un turno nuevo
POST /cash/close	CERRAR_CAJA	CashResponse	409 si no cuadra y no hay motivo
GET /cash/next-number	ABRIR_CAJA	NextNumber	Sugerencia: cuenta sobre las cajas

Se eliminaron dos endpoints de V3, que ya no significaban nada: POST /cash (creaba un corte vacío) y GET /cash/available (cajas nunca abiertas). El segundo era especialmente peligroso: su criterio (openedAt IS NULL) era correcto en V3 pero en V4 significaba “cajas que nunca se abrieron”, que después del primer turno de cada caja no sería ninguna.

7. Lo que quedó hecho y cómo se verifica

Área	Cambio	Estado
V4__cajas_reutilizables.sql	Crea cash_boxes, migra los datos, suelta el UNIQUE, agrega la FK	Hecho
Entidad	CashBoxEntity nueva; CashRegisterEntity gana cashBox	Hecho
Repositorios	CashBoxRepository nuevo; consultas por caja	Hecho
DTOs	CashBoxRequest y CashBoxResponse	Hecho
Service	open reescrito; CRUD de cajas; desactiveBox y deleteBox	Hecho
Controller	6 endpoints de cajas; 2 de V3 eliminados	Hecho
Tests	Reescritos de V3 a V4; +tests de reapertura, borrado condicional y acumulados en cero	258/258
Frontend	Interfaces, service, facade y el modal “Ver cajas” con su CSS	ng build OK

7.1 Cómo verificarlo a mano

- Registrar “CAJA 1” desde “Ver cajas”. Debe quedar con 0 turnos.
- Abrir la caja con 500. Vender algo en efectivo, por 200.

- Intentar borrar “CAJA 1”: debe salir el 409 con “dala de baja”, y en la tabla el botón debe ser “Dar de baja”, no “Borrar”.
- Cerrar con 700: cuadra exacto, no guarda motivo. Con 650: 409 y pide motivo; con motivo, cierra y guarda la diferencia de -50.
- Volver a abrir “CAJA 1” al día siguiente: debe permitirlo. El selector la ofrece porque no tiene turno abierto.
- En “Ver cajas”, CAJA 1 debe mostrar 2 turnos. Es la prueba visible de que el modelo nuevo funciona.

Si el frontend no encuentra los tests

ng test sigue diciendo “No test files found”. No es un defecto del código: el path del repo tiene paréntesis (Sistema de ventas (comida)) y rompen el glob de vitest. Está documentado en el AGENTS.md del frontend desde 2026-09-30. Para correrlos, hay que mover el proyecto a una ruta sin paréntesis.

8. En una frase

V4 no agregó una función nueva: **arregló un error de modelo**. La app decía tener una caja por fila de corte, cuando en un negocio real hay dos cajas y muchos turnos. Separar `cash_boxes` de `cash_registers` hizo que las cosas que son distintas dejaran de pelear entre sí: el número único pasó a estar donde corresponde (en las cajas), y el crecimiento diario pasó a estar en la tabla que crece (los turnos).

Los seis errores no fueron seis descuidos. Cinco fueron consecuencia directa de cambiar un modelo que dos clases habían fingido ser el mismo: un ! perdido, un nombre derivado que apuntaba al lado equivocado de la relación, un fondo sembrado en dos columnas. Y el sexto —el que no daba ningún error— era el peor de todos, porque un corte de caja que cuadra mal es un sistema que miente sobre el dinero.